Data Compression Explained

Matt Mahoney

Copyright (C) 2010-2012, Dell, Inc. You are permitted to copy and distribute material from this book provided (1) any material you distribute includes this license, (2) the material is not modified, and (3) you do not charge a fee or require any other considerations for copies or for any works that incorporate material from this book. These restrictions do not apply to normal “fair use,” defined as cited quotations totaling less than one page. This book may be downloaded without charge from mattmahoney.net/dc/dce.html.

Last update: Apr. 15, 2013.

About this Book

This book is for the reader who wants to understand how data compression works, or who wants to write data compression software. Prior programming ability and some math skills will be needed.

This book is intended to be self contained. Sources are linked when appropriate, but you don’t need to click on them to understand the material.

---

1. Information Theory

Data compression is the art of reducing the number of bits needed to store or transmit data. Compression can be either lossless or lossy. Losslessly compressed data can be decompressed to exactly its original value. An example is 1848 Morse Code. Each letter of the alphabet is coded as a sequence of dots and dashes. The most common letters in English like E and T receive the shortest codes. The least common like J, Q, X, and Z are assigned the longest codes.

All data compression algorithms consist of at least a model and a coder (with optional preprocessing transforms). A model estimates the probability distribution (E is more common than Z). The coder assigns shorter codes to

the more likely symbols. There are efficient and optimal solutions to the coding problem. However, optimal modeling has been proven not computable. Modeling (or equivalently, prediction) is both an artificial intelligence (AI) problem and an art.

Lossy compression discards “unimportant” data, for example, details of an image or audio clip that are not perceptible to the eye or ear. An example is the 1953 NTSC standard for broadcast color TV, used until 2009. The human eye is less sensitive to fine detail between colors of equal brightness (like red and green) than it is to brightness (black and white). Thus, the color signal is transmitted with less resolution over a narrower frequency band than the monochrome signal.

Lossy compression consists of a transform to separate important from unimportant data, followed by lossless compression of the important part and discarding the rest. The transform is an AI problem because it requires understanding what the human brain can and cannot perceive.

Information theory places hard limits on what can and cannot be compressed losslessly, and by how much:

1. There is no such thing as a “universal” compression algorithm that is guaranteed to compress any input, or even any input above a certain size. In particular, it is not possible to compress random data or compress recursively.
2. Given a model (probability distribution) of your input data, the best you can do is code symbols with probability p using log2(1/p) bits. Efficient and optimal codes are known.
3. Data has a universal but uncomputable probability distribution. Specifically, any string x has probability (about) 2^-|M| where M is the shortest possible description of x, and |M| is the length of M in bits, almost independent of the language in which M is written. However there is no general procedure for finding M or even estimating |M| in any language. There is no algorithm that tests for randomness or tells you whether a string can be compressed any further.

1.1. No Universal Compression

This is proved by the counting argument. Suppose there were a compression algorithm that could compress all strings of at least a certain size, say, n bits. There are exactly 2^n different binary strings of length n. A universal

compressor would have to encode each input differently. Otherwise, if two inputs compressed to the same output, then the decompresser would not be able to decompress that output correctly. However there are only 2^n - 1 binary strings shorter than n bits.

In fact, the vast majority of strings cannot be compressed by very much. The fraction of strings that can be compressed from n bits to m bits is at most 2^(m-n). For example, less than 0.4% of strings can be compressed by one byte.

Every compressor that can compress any input must also expand some of its input. However, the expansion never needs to be more than one symbol. Any compression algorithm can be modified by adding one bit to indicate that the rest of the data is stored uncompressed.

The counting argument applies to systems that would recursively compress their own output. In general, compressed data appears random to the algorithm that compressed it so that it cannot be compressed again.

1.2. Coding is Bounded

Suppose we wish to compress the digits of pi, e.g. “3141592653589793238462643…”. Assume our model is that each digit occurs with probability 0.1, independent of any other digits. Consider 3 possible binary codes:

```
Digit BCD   Huffman Binary
----  ----   ----   ----
  0   0000   000    0
  1   0001   001    1
  2   0010   010    10
  3   0011   011    11
  4   0100   100    100
  5   0101   101    101
  6   0110   1100   110
  7   0111   1101   111
  8   1000   1110   1000
  9   1001   1111   1001
---   ----   ----   ----
bpc   4.0    3.4    not valid
```

Using a BCD (binary coded decimal) code, pi would be encoded as 0011 0001 0100 0001 0101... (Spaces are shown for readability only). The compression ratio is 4 bits per character (4 bpc). If the input was ASCII text, the output would be compressed 50%. The decompresser would decode the data by dividing it into 4 bit strings.

The Huffman code would code pi as 011 001 100 001 101 1111... The decoder would read bits one at a time and decode a digit as soon as it found a match in the table (after either 3 or 4 bits). The code is uniquely decodable because no code is a prefix of any other code. The compression ratio is 3.4 bpc.

The binary code is not uniquely decodable. For example, 111 could be decoded as 7 or 31 or 13 or 111.

There are better codes than the Huffman code given above. For example, we could assign Huffman codes to pairs of digits. There are 100 pairs each with probability 0.01. We could assign 6 bit codes (000000 through 011011) to 00 through 27, and 7 bits (0111000 through 1111111) to 28 through 99. The average code length is 6.72 bits per pair of digits, or 3.36 bpc. Similarly, coding groups of 3 digits using 9 or 10 bits would yield 3.3253 bpc.

Shannon and Weaver (1949) proved that the best you can do for a symbol with probability p is assign a code of length log2(1/p). In this example, log2(1/0.1) = 3.3219 bpc.

Shannon defined the expected information content or equivocation (now called entropy) of a random variable X as its expected code length. Suppose X may have values X1, X2,... and that each Xi has probability p(i). Then the entropy of X is H(X) = E[log2 1/p(X)] = Sum p(i) log2 1/p(i). For example, the entropy of the digits of pi, according to our model, is 10 x (0.1 log2 1/0.1) = 3.3219 bpc. There is no smaller code for this model that could be decoded unambiguously.

The information content of a set of strings is at most the sum of the information content of the individual strings. If X and Y are strings, then H(X,Y) <= H(X) + H(Y). If they are equal, then X and Y are *independent*. Knowing one string would tell you nothing about the other.

The *conditional entropy* H(X|Y) = H(X,Y) - H(Y) is the information content of X given Y. If X and Y are independent, then H(X|Y) = H(X).

If X is a string of symbols x1x2…xn, then by the chain rule, p(X) may be expressed as a product of the sequence of symbol predictions conditioned on previous symbols: p(X) = prod p(xi | x1..i-1). Likewise, the information content H(X) of random string X is the sum of the conditional entropies of each symbol given the previous symbols: H(X) = sum H(xi | x1..i-1).

Entropy is both a measure of uncertainty and a lower bound on expected compression. The entropy of an information source is the expected limit to which you can compress it. There are efficient coding methods, such as arithmetic codes, which are for all practical purposes optimal in this sense. It should be emphasized, however, that entropy can only be calculated for a known probability distribution. But in general, the model is not known.

1.3. Modeling is Not Computable

We modeled the digits of pi as uniformly distributed and independent. Given that model, Shannon's coding theorem places a hard limit on the best compression that could be achieved. However, it is possible to use a better model. The digits of pi are not really random. The digits are only unknown until you compute them. An intelligent compressor might recognize the digits of pi and encode it as a description meaning "the first million digits of pi", or as a program that reconstructs the data when run. With our previous model, the best we could do is (10^6 log2 10)/8 ~ 415,241 bytes. Yet, there are very small programs that can output pi, some as small as 52 bytes.

The counting argument says that most strings are not compressible. So it is a rather remarkable fact that most strings that we care about, for example English text, images, software, sensor readings, and DNA, are in fact compressible. These strings generally have short descriptions, whether they are described in English or as a program in C or x86 machine code.

Solomonoff (1960, 1964), Kolmogorov (1965), and Chaitin (1966) independently proposed a universal a-priori probability distribution over strings based on their minimum description length. The algorithmic probability of a string x is defined as the fraction of random programs in some language L that output x, where each program M is weighted by 2^-|M| and |M| is the length of M in bits. This probability is dominated by the shortest such program. We call this length the Kolmogorov complexity K_L(x) of x.

Algorithmic probability and complexity of a string x depend on the choice of language L, but only by a constant that is independent of x. Suppose that M1 and M2 are encodings of x in languages L1 and L2 respectively. For example, if L1 is C++, then M1 would be a program in C++ that outputs x. If L2 is English, the M2 would be a description of x with just enough detail that it would allow you to write x exactly. Now it is possible for any pair of languages to write in one language a compiler or interpreter or rules for understanding the other language. For example, you could write a description of the C++ language in English so that you could (in theory) read any C++ program and predict its output by "running" it in your head. Conversely, you could (in theory) write a program in C++ that input an English language description and translated it into C++. The size of the language description or compiler does not depend on x in any way. Then for any description M1 in any language L1 of any x, it is possible to find a description M2 in any other language L2 of x by appending to M1 a fixed length description of L1 written in L2.

It is not proven that algorithmic probability is a true universal prior probability. Nevertheless it is widely accepted on empirical grounds because of its success in sequence prediction and machine learning over a wide range of data types. In machine learning, the minimum description length principle of choosing the simplest hypothesis consistent with the training data applies to a wide range of algorithms. It formalizes Occam's Razor. Occam noted in the 14th century, that (paraphrasing) "the simplest answer is usually the correct answer." Occam's Razor is universally applied in all of the sciences because we know from experience that the simplest or most elegant (i.e. shortest) theory that explains the data tends to be the best predictor of future experiments.

To summarize, the best compression we can achieve for any string x is to encode it as the shortest program M in some language L that outputs x. Furthermore, the choice of L becomes less important as the strings get longer. All that remains is to find a procedure that finds M for any x in some language L. However, Kolmogorov proved that there is no such procedure in any language. In fact, there is no procedure that just gives you |M|, the length of the shortest possible description of x, for all x. Suppose there were. Then it would be possible to describe "the first string that cannot be described in less than a million bits" leading to the paradox that we had just done so. (By "first," assume an ordering over strings from shortest to longest, breaking ties lexicographically).

Nor is there a general test for randomness. Li and Vitanyi (2007) give a simple proof of a stronger variation of Godel's first incompleteness theorem, namely that in any consistent, formal system (a set of axioms and rules of inference) powerful enough to describe statements in arithmetic, there is at least one true statement that cannot be proven. Li and Vitanyi prove that if the system is sound (you can't prove any false statements) then there are an infinite number of true but unprovable statements. In particular, there are only a finite number of statements of the form "x is random" (K(x) >= |x|) that can be proven, out of an infinite number of possible finite strings x. Suppose otherwise. Then it would be possible to enumerate all proofs (lists of axioms and applications of the rules of inference) and describe "the string x such that it is the first to be proven to be a million bits long and random" in spite of the fact that we just gave a short description of it. If F describes any formal system, then the longest string that can be proven to be random is never much longer than F, even though there are an infinite number of longer strings and most of them are random.

Nor is there a general test to determine whether the compression of a string can be improved any further, because that would be equivalent to testing whether the compressed string is random.

Because determining the length of the shortest descriptions of strings is not computable, neither is optimal compression. It is not hard to find difficult cases. For example, consider the short description "a string of a million zero bytes compressed with AES in CBC mode with key 'foo'". To any program that does not know the key, the data looks completely random and incompressible.

1.4. Compression is an Artificial Intelligence Problem

Prediction is intuitively related to understanding. If you understand a sequence, then you can predict it. If you understand a language, then you could predict what word might appear next in a paragraph in that language. If you understand an image, then you could predict what might lie under portions of the image that are covered up. Alternatively, random data has no meaning and is not predictable. This intuition suggests that prediction or compression could be used to test or measure understanding. We now formalize this idea.

Optimal compression, if it were computable, would optimally solve the artificial intelligence (AI) problem under two vastly different definitions of "intelligence": the Turing test (Turing, 1950), and universal intelligence (Legg and Hutter, 2006).

Turing first proposed a test for AI to sidestep the philosophically difficult question (which he considered irrelevant) of whether machines could think. This test, now known as the Turing test, is now widely accepted. The test is a game played by two humans who have not previously met and the machine under test. One human (the judge) communicates with the other human (the confederate) and with the machine through a terminal. Both the confederate and the machine try to convince the judge that each is human. If the judge cannot guess correctly which is the machine 70% of the time after 10 minutes of interaction, then the machine is said to have AI. Turing gave the following example of a possible dialogue:

> Q: Please write me a sonnet on the subject of the Forth Bridge. A: Count me out on this one. I never could write poetry. Q: Add 34957 to 70764. A: (Pause about 30 seconds and then give as answer) 105621. Q: Do you play chess? A: Yes. Q: I have K at my K1, and no other pieces. You have only K at K6 and R at R1. It is your move. What do you play? A: (After a pause of 15 seconds) R-R8 mate.

It should be evident that compressing transcripts like this requires the ability to compute a model of the form p(A|Q) = p(QA)/P(Q) where Q is the context up to the current question, and A is the response. But if a model could make such predictions accurately, then it could also generate responses indistinguishable from that of a human.

Predicting transcripts is similar to the problem of predicting ordinary written language. It requires in either case vast, real-world knowledge. Shannon (1950) estimated that the information content of written case-insensitive English without punctuation is 0.6 to 1.3 bits per character, based on experiments in which human subjects guessed successive characters in text with the help of letter n-gram frequency tables and dictionaries. The uncertainty is due not so much to variation in subject matter and human skill as it is due to the fact that different probability assignments lead to the same observed guessing sequences. Nevertheless, the best text compressors are only now compressing near the upper end of this range.

Legg and Hutter proposed the second definition, universal intelligence, to be far more general than Turing's human intelligence. They consider the problem of reward-seeking agents in completely arbitrary environments described by random programs. In this model, an agent communicates with an environment by sending and receiving symbols. The environment also sends a reinforcement or reward signal to the agent. The goal of the agent is to maximize accumulated reward. Universal intelligence is defined as the expected reward over all possible environments, where the probability of each environment described by a program M is algorithmic, proportional to 2^-|M|. Hutter (2004, 2007) proved that the optimal (but not computable) strategy for the agent is to guess after each input that the distribution over M is dominated by the shortest program consistent with past observation.

Hutter calls this strategy AIXI. It is, of course, just our uncomputable compression problem applied to a transcript of past interaction. AIXI may also be considered a formal statement and proof of Occam's Razor. The best predictor of the future is the simplest or shortest theory that explains the past.

1.5. Summary

There is no such thing as universal compression, recursive compression, or compression of random data.

Most strings are random. Most meaningful strings are not.

Compression = modeling + coding. Coding is a solved problem. Modeling is provably not solvable.

Compression is both an art and an artificial intelligence problem. The key to compression is to understand the data you want to compress.

[Note: this PDF currently covers the "About this Book" section and Section 1 (Information Theory) of the full book. The complete text runs through Section 6 (Lossy Compression), the Conclusion, Acknowledgements, and References. I can continue converting the remaining sections — Benchmarks, Coding, Modeling, Transforms, and Lossy Compression — in follow-up passes if you'd like the full book as a single PDF.]